

Aufgabe 1 – Datenabhängigkeiten und Pipeline-Konflikte

Code:

main:

```
i1    addi    t0,    zero, 10
i2    addi    t1,    zero, 20
i3    add     t2,    t0,    t1
i4    and     t0,    t1,    t0
i5    beq     t0,    zero, skip
i6    addi    t3,    zero, 1
      skip:
i7    add     t3,    t2,    t3
```

a) Datenabhängigkeiten:

t0:

i1 und i3: RAW
i1 und i4: RAW, WAW
i1 und i5: RAW
i3 und i4: WAR, (RAR)
i3 und i5: - (both read)
i4 und i5: RAW, (RAR)

t1:

i2 und i3: RAW
i2 und i4: RAW
i3 und i4: - (both read)

t2:

i3 und i7: RAW

t3:

i6 und i7: RAW, WAW

b) Konflikte:

	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	add	addi	addi		
4	and	add	addi	addi	
5	beq	and	add	addi	addi
6	addi	beq	and	add	addi
7	add	addi	beq	and	add
8		add	addi	beq	and
9			add	addi	beq
10				add	addi
11					add

Datenkonflikte:

- ➔ Für i3-Decode müssen i1 und i2 mit Writeback fertig sein
- ➔ Für i4-Decode müssen i1 und i2 mit Writeback fertig sein
- ➔ Für i5-Decode müssen i4 und i1 mit Writeback fertig sein
- ➔ Wenn bei i5 Branch nicht genommen wurde:
 - > für i7-Decode muss i6 mit Writeback fertig sein

Kontrollkonflikte:

- ➔ Wenn bei i5 Branch genommen wird -> i6 sollte nicht geladen sein (i7 in diesem Fall ok, da nur i6 übersprungen wird)

Computer Architecture

Übung 06

Aufgabe 02cd - Datenabhängigkeiten und Pipeline-Konflikte

Teammitglieder

- Aleksandar Stojanović, K12411325
 - Annika Schmidthaler, K12411307
 - Benedikt Zöchmann, K12410383
-

Betrachte das folgende RISC-V Programm und nimm an, dass es auf einem RISC-V Prozessor (siehe Abbildung 1) ausgeführt werden soll.

```
main:
    addi t0, zero, 10
    addi t1, zero, 20
    add t2, t0, t1
    and t0, t1, t0
    beq t0, zero, skip
    addi t3, zero, 1
skip:
    add t3, t2, t3
```

c) Behebe alle Pipeline-Konflikte durch Einfügen der minimalen Anzahl an NOP-Befehlen.

d) Minimiere die Anzahl der NOP-Befehle durch Umordnen der Befehle (ohne die Semantik des Programms zu verändern). Nimm dabei an, dass der Prozessor auch das Löschen (flushen) von Pipeline-Registern unterstützt.

Notiz: Wenn man sich die Immediate Werte anschaut, kann der Fall "Nicht Springen" nicht vorkommen, da die Bedingung von `beq` immer true ist. Vollständigkeitshalber (und um mit der in der Übung gezeigten Methode übereinzustimmen) wird trotzdem dieser Fall hier berücksichtigt.

Moodlepost zur Notwendigkeit der zwei Fälle: [20255366009/21/25/26/27 | JKU Moodle](https://moodle.jku.at/question/view.php?id=20255366009)

c.)

Ausgangsposition

	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	add	addi	addi		
4	and	add	addi	addi	
5	beq	and	add	addi	addi
6	addi	beq	and	add	addi
7	add	addi	beq	and	add
8		add	addi	beq	and
9			add	addi	beq
10				add	addi
11					and

Einfügen von NOPs:

kein Sprung	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	nop	addi	addi		
4	nop	nop	addi	addi	
5	nop	nop	nop	addi	addi
6	add	nop	nop	nop	addi
7	and	add	nop	nop	nop
8	nop	and	add	nop	nop
9	nop	nop	and	add	nop
10	nop	nop	nop	and	add
11	beq	nop	nop	nop	and
12	nop	beq	nop	nop	nop
13	nop	nop	beq	nop	nop
14	addi	nop	nop	beq	nop
15	nop	addi	nop	nop	beq
16	nop	nop	addi	nop	nop
17	nop	nop	nop	addi	nop
18	add	nop	nop	nop	addi
19		add	nop	nop	nop
20			add	nop	nop
21				add	nop
22					add

Man benötigt 11 NOPs

Sprung	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	nop	addi	addi		
4	nop	nop	addi	addi	
5	nop	nop	nop	addi	addi
6	add	nop	nop	nop	addi
7	and	add	nop	nop	nop
8	nop	and	add	nop	nop
9	nop	nop	and	add	nop
10	nop	nop	nop	and	add
11	beq	nop	nop	nop	and
12	nop	beq	nop	nop	nop
13	nop	nop	beq	nop	nop
14	add	nop	nop	beq	nop
15		add	nop	nop	beq
16			add	nop	nop
17				add	nop
18					add

Man benötigt 8 NOPs

d.)

Originalform

```

main:
    addi t0, zero, 10
    addi t1, zero, 20
    add t2, t0, t1
    and t0, t1, t0
    beq t0, zero, skip
    addi t3, zero, 1
skip:
    add t3, t2, t3

```

Abgeänderte Form unter Berücksichtigung der Abhängigkeiten:

```
# Kein Sprung
main:
    addi t0, zero, 10
    addi t1, zero, 20
    addi t3, zero, 1
    and t0, t1, t0
    add t2, t0, t1
    beq t0, zero, skip
skip:
    add t3, t2, t3
```

Anmerkung: Eigentlich dürfte man nicht `addi t3, t2, t3` verschieben, da wie schon gesagt der Pfad "Nicht Springen" mit den gegebenen Werten nicht auftreten kann. Da wir aber schon zw. den Szenarien "Springen" und "Nicht Springen" unterscheiden, ist es in diesem Fall zulässig.

```
# Sprung
main:
    addi t0, zero, 10
    addi t1, zero, 20
    and t0, t1, t0
    add t2, t0, t1
    beq t0, zero, skip
    addi t3, zero, 1
skip:
    add t3, t2, t3
```

kein Sprung	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	addi	addi	addi		
4	nop	addi	addi	addi	
5	nop	nop	addi	addi	
6	and	nop	nop	addi	addi
7	add	and	nop	nop	addi
8	nop	add	and	nop	nop
9	nop	nop	add	and	nop
10	beq	nop	nop	add	and
11	nop	beq	nop	nop	add
12	nop	nop	beq	nop	nop
13	add	nop	nop	beq	nop
14		add	nop	nop	beq
15			add	nop	nop
16				add	nop

Durch die Umordnung können wir die benötigten NOPs auf 6 reduzieren.

Sprung	Fetch	Decode	Execute	Memory	Writeback
1	addi				
2	addi	addi			
3	nop	addi	addi		
4	nop	nop	addi	addi	
5	nop	nop	nop	addi	addi
6	and	nop	nop	nop	addi
7	add	and	nop	nop	nop
8	nop	add	and	nop	nop
9	nop	nop	add	and	nop
10	beq	nop	nop	add	and
11	nop	beq	nop	nop	add
12	nop	nop	beq	nop	nop
13	add	nop	nop	beq	nop
14		add	nop	nop	beq
15			add	nop	nop
16				add	nop
17					add

Durch die Umordnung können wir die benötigten NOPs auf 7 reduzieren.
